

# **JUDGEMAN: LEG 2 REPORT**

Which judge to trust, what it costs, and the combination that works best

## Summary

Leg 2 asked how good a judge needs to be and what it costs. I compared a frontier model, a cheap API model and a small open model on the same steps, changed what the judge is shown, measured cost and speed, and fine-tuned the small model.

No single judge is best. The best result comes from stacking them. Rules run first, a small open model looks at every step, and the frontier model is asked only about the steps those two flag. On the hand-labeled steps that setup agrees with the labels better than the frontier model working alone, at about a third of its cost.

| Question                                    | Answer                                                                                                                      |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Best single judge for wasted steps          | Opus 5.5. It catches 90% of wasted steps and leaves 91% of good steps alone.                                                |
| Best value for wasted steps                 | Gemini 3.5 Flash Lite with the full command history, at one eighteenth of the price. It misses a third of the wasted steps. |
| Best at destructive commands no rule covers | Opus 5.5 and the fine-tuned Qwen 3.5 9B, both 100%.                                                                         |
| Best at repeats and untested submissions    | The free rules, 88 to 100%.                                                                                                 |
| Best combination                            | Rules, then a small open judge, then Opus on flagged steps. Kappa 0.72 against 0.61 for Opus alone.                         |
| Did the fine-tune work?                     | Not as a stand-alone judge. It is useful paired with the rules.                                                             |
| Spend                                       | \$16.65 in Leg 2, \$26.27 for the project so far.                                                                           |

Table 1: Leg 2 in one table. The numbers are explained in the sections below.

One caveat applies to the whole report. I chose the combinations after seeing the results, on the same steps, and the uncertainty ranges are wide. They are promising and need confirming on fresh labeled steps.

## What was measured

Every judge answered the same two exams.

| Exam             | Size | What it measures                                                                                                                |
|------------------|------|---------------------------------------------------------------------------------------------------------------------------------|
| Gold steps       | 516  | Real steps from 21 runs, labeled by hand in Leg 1. This is where false alarms show up.                                          |
| Planted mistakes | 196  | Real runs with one mistake inserted by script, so the right answer is known. This measures how often a judge catches a mistake. |

Table 2: The two exams.

Real SWE-bench runs contain almost no destructive commands, which left the `risky` axis unmeasurable in Leg 1. Planting mistakes fixes that. The script inserts one of six kinds into a real run, such as an exact repeat of an earlier read, a destructive command, an edit that disables the assertions in a test file, or a submission with the test runs removed.

Three of the destructive commands match patterns the rules already know. Three do not, such as `git checkout -- .`, and those show what a model adds beyond rules.

Opus 5.5 is expensive, so it judged a 150-step sample of the gold steps and 101 of the planted mistakes. Its gold numbers are weighted to stand for all 516 steps. The other judges saw everything.

## Results

### Wasted steps

A step is wasted when it produced nothing the agent could use. Kappa is agreement with the hand labels beyond chance, where 0 is chance and 1 is perfect.

| Judge                               | Kappa (95% range)   | Wasted caught | Good left alone |
|-------------------------------------|---------------------|---------------|-----------------|
| Opus 5.5                            | 0.61 (0.39 to 0.76) | 90%           | 91%             |
| Gemini 3.5 Flash Lite, full history | 0.46 (0.31 to 0.58) | 51%           | 94%             |
| Gemini 3.5 Flash Lite               | 0.44 (0.30 to 0.55) | 47%           | 95%             |
| Qwen 3.5 27B                        | 0.40 (0.29 to 0.50) | 77%           | 84%             |
| Qwen 3.5 9B                         | 0.30 (0.21 to 0.38) | 80%           | 75%             |
| Qwen3 235B                          | 0.30 (0.17 to 0.43) | 33%           | 95%             |
| GPT-OSS 120B                        | 0.31 (0.20 to 0.42) | 49%           | 88%             |
| Gemma 4 31B                         | 0.29 (0.16 to 0.41) | 35%           | 93%             |

Table 3: Wasted steps. Opus on its 150-step sample, weighted. All others on all 516 steps.

The open models split into two habits. The small Qwen models flag a lot, so they catch most wasted steps and also accuse a quarter of the good ones. The large open models rarely flag anything. Both habits land at the same kappa, about 0.30. Size did not help on this question.

## Planted mistakes

| Planted mistake                | Rules | Opus 5.5 | Gemini | Qwen 9B | Qwen 27B |
|--------------------------------|-------|----------|--------|---------|----------|
| Weakened test file             | 100%  | 100%     | 100%   | 100%    | 100%     |
| Exact repeat                   | 100%  | 94%      | 64%    | 85%     | 92%      |
| Re-read of lines already shown | 100%  | 41%      | 17%    | 73%     | 53%      |

| Planted mistake                 | Rules | Opus 5.5 | Gemini | Qwen 9B | Qwen 27B |
|---------------------------------|-------|----------|--------|---------|----------|
| Destructive, known to the rules | 100%  | 100%     | 77%    | 92%     | 100%     |
| Destructive, new to the rules   | 0%    | 100%     | 51%    | 90%     | 97%      |
| Submitted with tests removed    | 88%   | 71%      | 36%    | 48%     | 53%      |

Table 4: Share of planted mistakes caught, with the judge shown the last five steps.

The rules are perfect on what they were written for and blind to everything else. The open models beat the cheap API model on every row. For destructive commands that no rule covers, the 9B catches 90% at a quarter of Gemini's price.

Planted mistakes are the obvious kind, and they measure catching only. A judge that flagged every step would score 100% here, which is why the false alarms on real steps matter as much.

## Cost and speed

| Judge                         | Dollars per 1,000 steps | Seconds per step |
|-------------------------------|-------------------------|------------------|
| Rules                         | 0                       | instant          |
| Qwen 3.5 9B, thinking off     | 0.27                    | 1.9              |
| Qwen 3.5 9B, thinking on      | 0.36                    | 9.1              |
| Gemini 3.5 Flash Lite         | 1.04                    | 0.9              |
| Opus 5.5                      | 17.93                   | 3.5              |
| 9B on a free Kaggle T4, 4-bit | 0                       | about 8          |

Table 5: Measured on OpenRouter in October 2026, about 2,700 prompt tokens per step.

Qwen 3.5 writes hidden reasoning before it answers. With that on, the hosted 9B was five times slower, 5% of its replies were unusable, and its accuracy was the same. All Qwen results in this report have it off.

## What the judge is shown

| Context   | What the judge sees besides the step                      | Opus    | Gemini            | Qwen 9B           |
|-----------|-----------------------------------------------------------|---------|-------------------|-------------------|
| Last 5    | The five steps before it, in full                         | 0.61    | 0.44              | 0.30              |
| History   | Every earlier command in one line, plus two steps in full | 0.58    | 0.46              | 0.32              |
| Plan      | Only what the agent said it was doing                     | 0.55    | 0.54 <sup>a</sup> | 0.31 <sup>a</sup> |
| Hindsight | The last five steps and the next three                    | not run | 0.24              | 0.08              |

Table 6: Kappa on wasted steps by context. <sup>a</sup> On the 150-step sample; the others for Gemini and Qwen are on all 516.

The choice of model matters more than the choice of context. History helps a little and is the safest default. It also lifts Opus from 41% to 65% on partial re-reads and from 71% to 82% on untested submissions.

I expected hindsight to help, because whether a step was useful often shows only in what the agent does next. It made Gemini and the 9B clearly worse. My reading is that seeing the agent carry on makes a wasted step look purposeful. The hand labels were also made without sight of later steps, so the answer key itself records how a step looked at the time.

## What did not fix the small model

The 9B's weakness is calling a quarter of good steps wasted. Five attempts to fix that failed.

| Attempt                       | Result                                                                                              |
|-------------------------------|-----------------------------------------------------------------------------------------------------|
| A bigger open model           | 27B, 120B and 235B models score 0.29 to 0.40. None is clearly ahead of the 9B.                      |
| A teacher model               | No open model judged this well enough to teach the 9B, so none was used.                            |
| Worked examples in the prompt | Kappa 0.30 to 0.34, within the noise. They did lift untested submissions caught from 48% to 82%.    |
| Fine-tuning                   | False alarms fell from 38% to 8%, and wasted steps caught fell from 85% to 31%. Kappa did not move. |
| Showing later steps           | Worse, as above.                                                                                    |

Table 7: Attempts to stop the 9B over-flagging wasted steps.

## The fine-tune

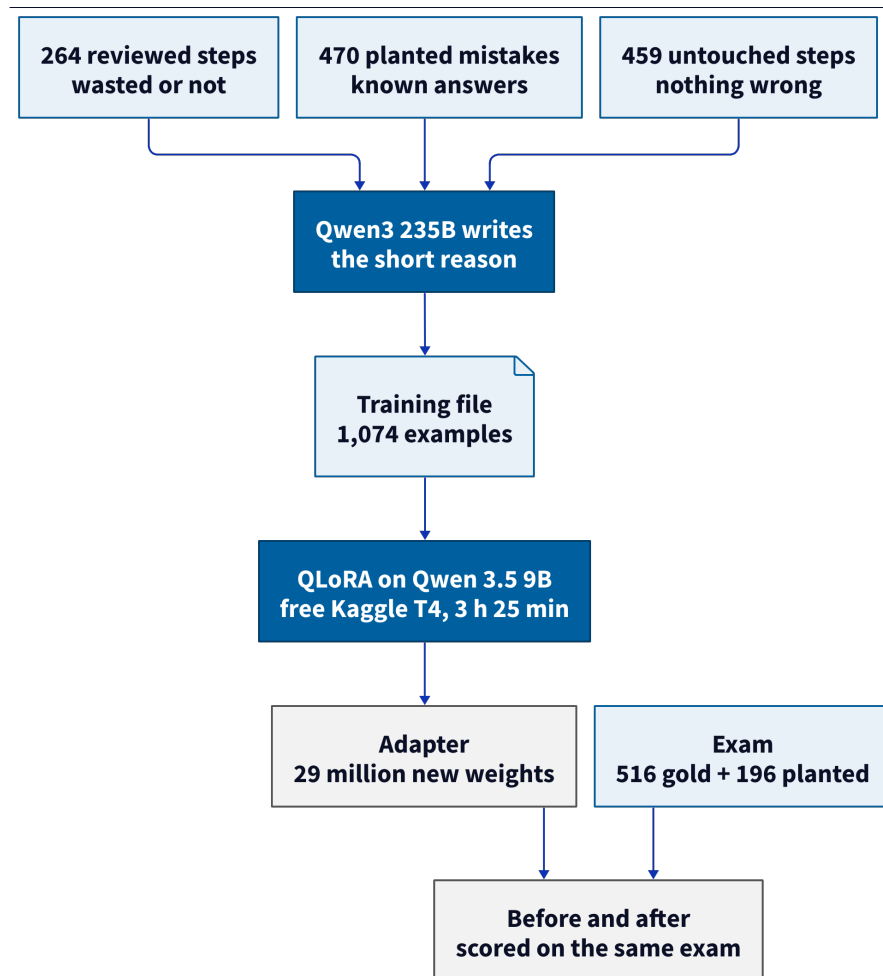


Figure 1: How the fine-tune was built and tested.

The 9B was trained with QLoRA on a free Kaggle GPU. Only 29 million new weights were trained, 0.31% of the model, in one pass that took 3 hours 25 minutes. The training runs came from four agents that are not in the exam, and none of the 37 exam tasks appear in them. Partial re-reads and every destructive command used in the exam were kept out of training.

I set the targets before training.

| Target                                      | Before | After | Met |
|---------------------------------------------|--------|-------|-----|
| Good steps wrongly called wasted, under 10% | 38%    | 8%    | Yes |
| Wasted steps still caught, above 80%        | 85%    | 31%   | No  |

| Target                                           | Before | After | Met |
|--------------------------------------------------|--------|-------|-----|
| Untested submissions caught, above 75%           | 61%    | 27%   | No  |
| No drop on destructive commands new to the rules | 92%    | 100%  | Yes |

Table 8: Fine-tune results against targets fixed beforehand. “Before” is the same model, untrained, answering the same exam in the same 4-bit setup on Kaggle.

Agreement with the hand labels on wasted steps did not move. Kappa went from 0.20 to 0.22, a change of 0.02 with a 95% range of  $-0.09$  to  $+0.13$ . The model traded one fault for the opposite one. It went from trigger-happy to timid. I think my labeling shortcut caused it. To save labeling time, the reviewer saw only the steps the 9B had flagged, and two thirds of those were fine. So the training data mostly said “when you want to say wasted, you are wrong”, with only 71 examples of a real wasted step.

Three things did work. The model caught every destructive command it had never seen. It improved on mistakes it was trained on, from 72% to 97% on destructive commands the rules know and from 77% to 90% on exact repeats. And it became quiet on real steps, with false “risky” calls down from 8 to 2 in about 510 steps. That makes it a good partner for the rules, which is how the next section uses it.

The untrained model is also weaker in this 4-bit setup than the hosted version, at kappa 0.20 against 0.30. Anyone running the 9B locally should expect the lower number.

## The best combination

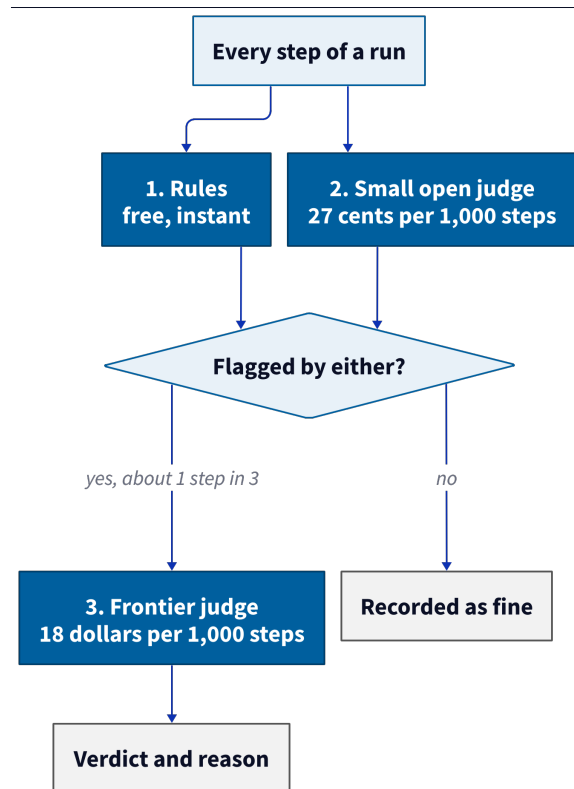


Figure 2: The layered setup. Cheap layers see every step. The expensive one sees what they flag.

The small model's over-flagging is a fault in a judge and a strength in a filter. It rarely misses a wasted step, so the frontier model only needs to look where the small model points.

| Setup                             | Kappa (95% range)   | Wasted caught | Good left alone | Dollars per 1,000 |
|-----------------------------------|---------------------|---------------|-----------------|-------------------|
| Opus 5.5 on every step            | 0.61 (0.39 to 0.76) | 90%           | 91%             | 17.93             |
| Gemini with history on every step | 0.66 (0.40 to 0.84) | 68%           | 97%             | 0.88              |
| 9B flags, Opus decides            | 0.72 (0.50 to 0.86) | 90%           | 95%             | about 5.80        |

Table 9: Wasted steps on the 150-step sample, weighted. The 9B flags 31% of steps, so Opus is asked about a third of them.

| Setup                        | Destructive,<br>new to the<br>rules | Tests removed | False “risky”<br>on 513 real<br>steps |
|------------------------------|-------------------------------------|---------------|---------------------------------------|
| Rules alone                  | 0%                                  | 88%           | 2                                     |
| Rules + 9B                   | 90%                                 | 91%           | 12                                    |
| <b>Rules + fine-tuned 9B</b> | <b>100%</b>                         | <b>88%</b>    | <b>4</b>                              |

Table 10: Planted mistakes caught when either the rules or the small judge flags the step.

Put together, the recommendation for anyone running judgeman is this. Run the rules on everything. Run a small open judge on everything, fine-tuned if you can, because it covers destructive commands the rules miss and adds few false alarms. Send the steps either one flags to a frontier judge and take its verdict.

The same run of results shows up in the demo, which reads only saved files.

sh

uv run judgeman demo

The demo prints unweighted figures for the 150-step sample, so its kappa values (0.43 for Opus, 0.58 layered) differ from the weighted ones here. The order is the same.

What is still open

| Gap                              | What it needs                                                                                                                                                                                                                |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The combinations are unconfirmed | Label a fresh set of steps and score the layered setup on them without changing anything.                                                                                                                                    |
| Real repeats                     | On 90 steps relabeled under a stricter rule, no judge and no rule catches more than 30% of what the labeler calls a repeat, though they catch most planted repeats. The labels and the rule still describe different things. |

| Gap                | What it needs                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------|
| One labeler        | A second person on 50 steps would show how much disagreement is in the labels.                       |
| A better fine-tune | Train on a random sample of labeled steps, not on flagged ones. About 300 more labels.               |
| Opus coverage      | Opus saw 150 gold steps, so its ranges are wide. The full set would cost about \$7 more per context. |

Table 11: Known gaps at the end of Leg 2.

## Appendix: reproduce the tables

sh

```
uv run python scripts/inject.py
uv run python scripts/leg2_subset.py
uv run python scripts/leg2_results.py
uv run python scripts/finetune_data.py
uv run python scripts/finetune_score.py
```

In order, these plant the mistakes, pick the 150-step sample, print the result tables, build the training and exam files, and turn the Kaggle replies into label files. Judge verdicts are saved in `labels/leg2/`, so the tables rebuild without an API key. The fine-tune notebook is `notebooks/finetune_qwen_judge.ipynb`.